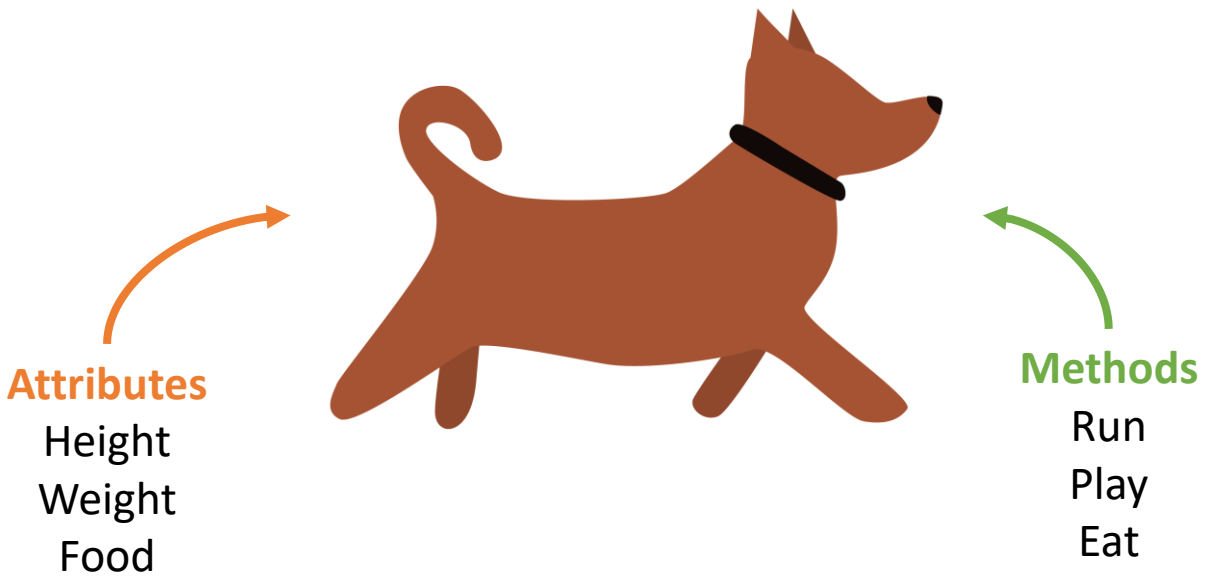
QUIZ

On CADT LMS

MOBILE DEVELOPMENT

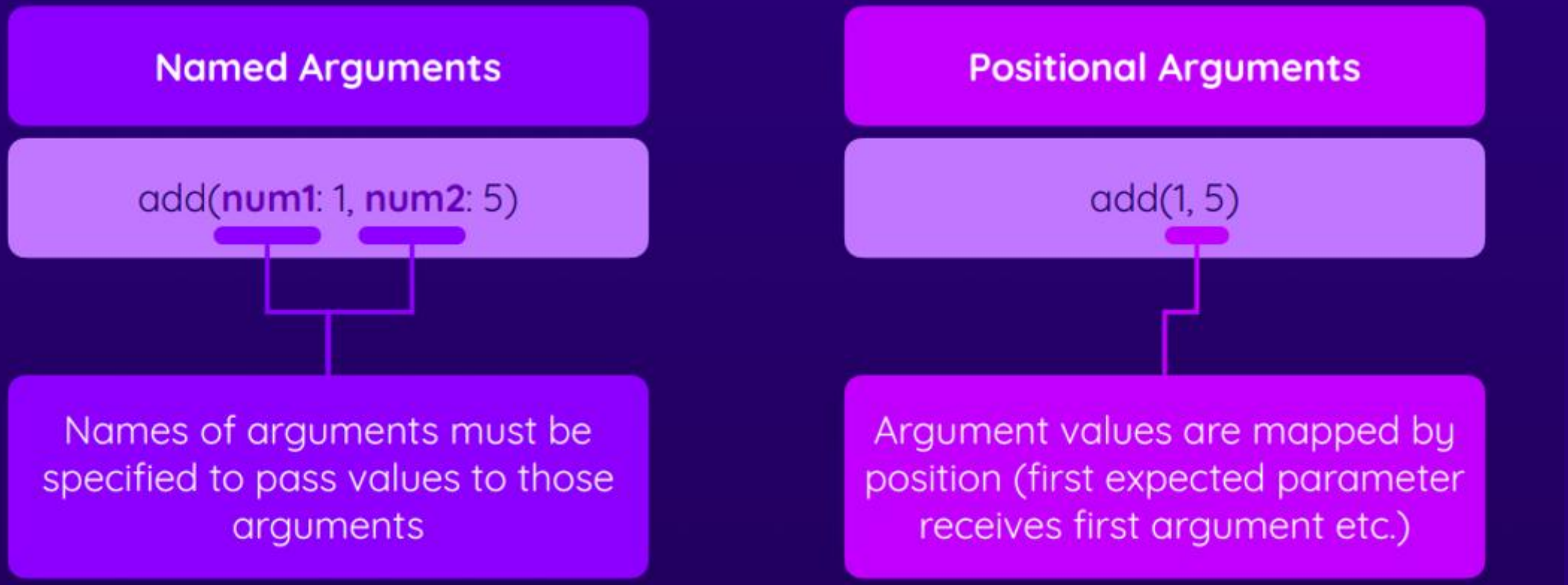
W2 – Dart – OOP





WE REVIEW

Dart arguments can be '**named**' or '**positional**'



WE REVIEW

Let’s implement together the bellow **function**

Description

Name	**printStudent**
Parameters	Name Age (default is 20) Address (optional) Score (a list of scores)
Return	void
Description	print the student information

Example

```
printStudent(name:"ronan“, scores: [20,15,45])
```

```
Student          : Ronan
Age              : 20
Address          : No address
Scores           : 15 20 45
```

ANSWER

```dart
void printStudent({
  required String name,
  int age = 20,
  String? address,
  required List<int> scores,
}) {
  // Name
  print("Student\t\t: $name");


  // Age
  print("Age\t\t: $age");


  // Address
  String addressValue;
  if (address != null) {
    addressValue = address;
  } else {
    addressValue = "No address";
  }
  print("Address\t\t: $addressValue");


  // Scores
  String scoreslabel = "";
  for (int score in scores) {
    scoreslabel += score.toString() + " ";
  }
  print("Scores\t\t: $scoreslabel");
}


void main() {
  printStudent(name: "Ronan", scores: [15, 20, 45]);
}
```

Named parameters (optional by default)

Required (not optional)

Default value, if not specified

Address is a **nullable type**

Nullable state must be checked

For-in loop on the **iterable** list of integer

EXPLAIN

4 ways to manage **nullable** data

```
type? data;
```

- **NULL ASSERTION** OPERATOR

```
print(data!);
```

I guarantee that data is NOT null

Treat it as a non null type.

- CLASSIC CONDITINAL BRANCH

```
if (data != null) {
  print(data);
} else {
  print("default value");
}
```

- TERNARY OPERATOR

```
print(data != null ? data : "default value");
```

- **NULL-COALESCING** OPERATOR

```
print(data?? "default value");
```



Session Objectives





JUNIOR LEVEL

- ✓ Create a **class** with **attributes** and **constructors**
- ✓ Add **methods**
- ✓ Build **aggregation of objects**
- ✓ Create **computed attributes** (get)



NINJA LEVEL

- ✓ Build **immutable** objects
- ✓ Create objects with **optional attributes**
- ✓ Create objects using **different named constructors**
- ✓ Overload the **operator +**



A CLASS

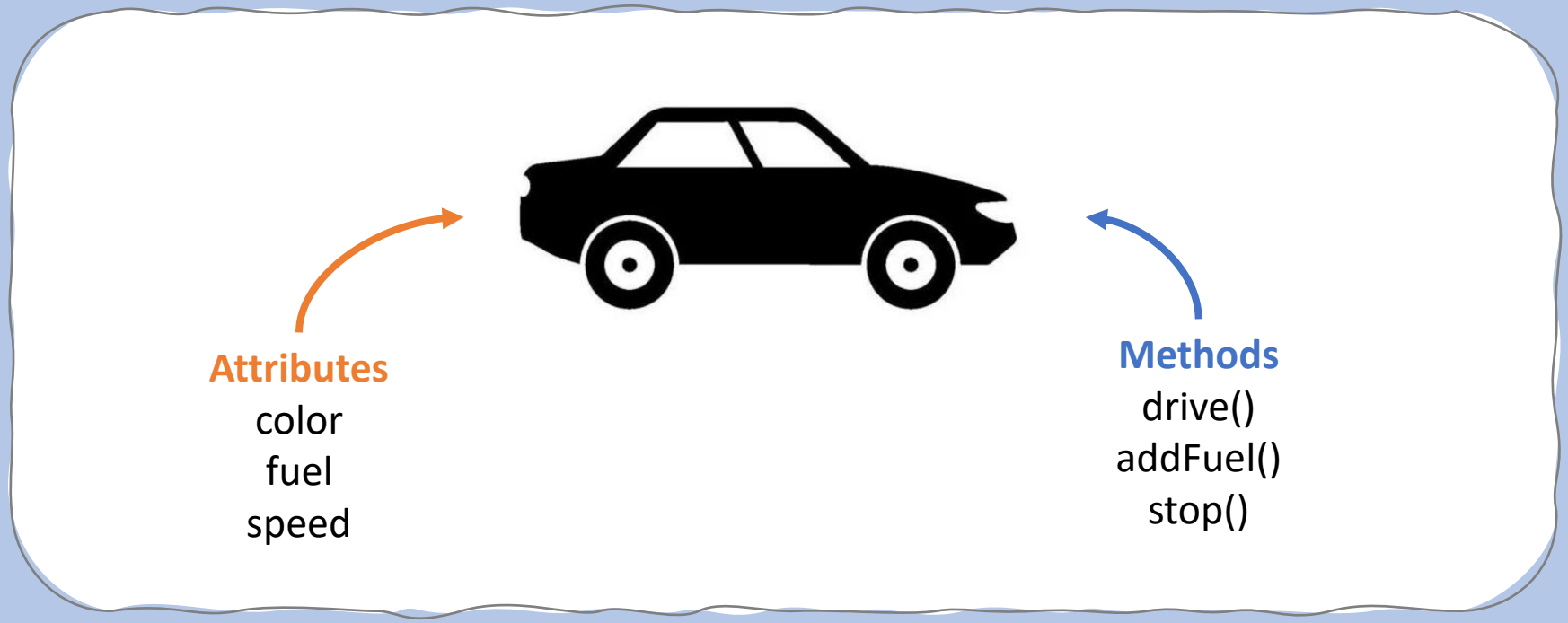
IS A

BLUEPRINT

FOR

OBJECTS

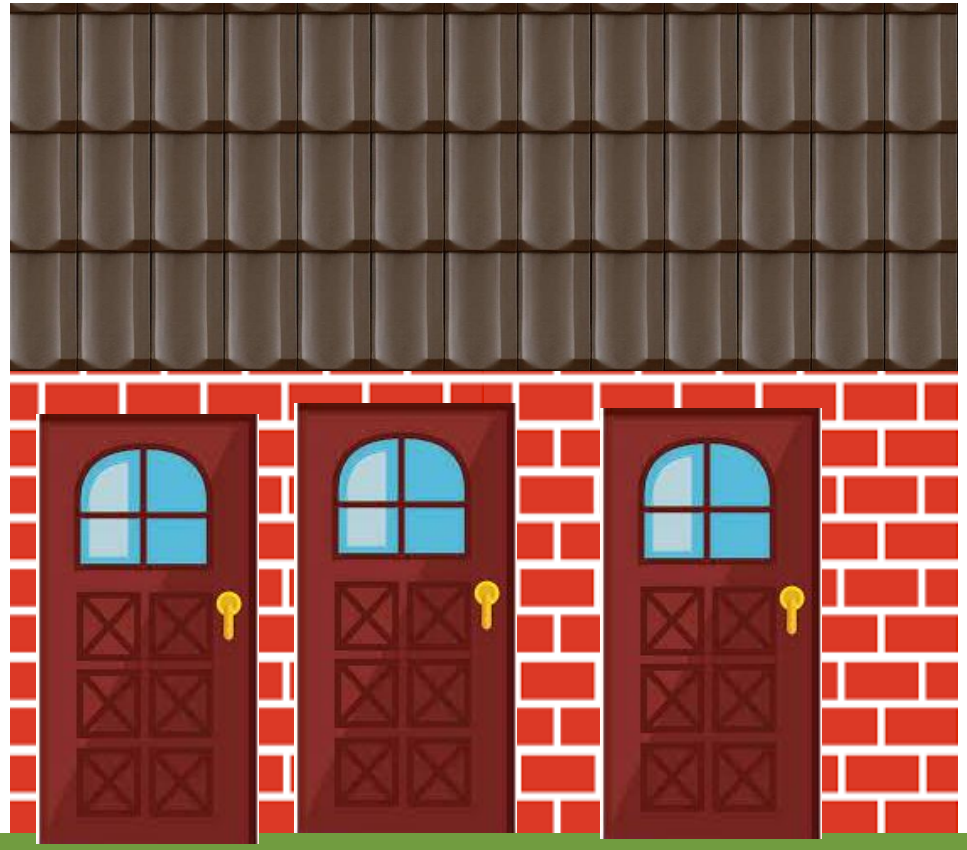
Dart is an **object-oriented language**



Thanks to classes, I can gather my car attributes and functions in a same place

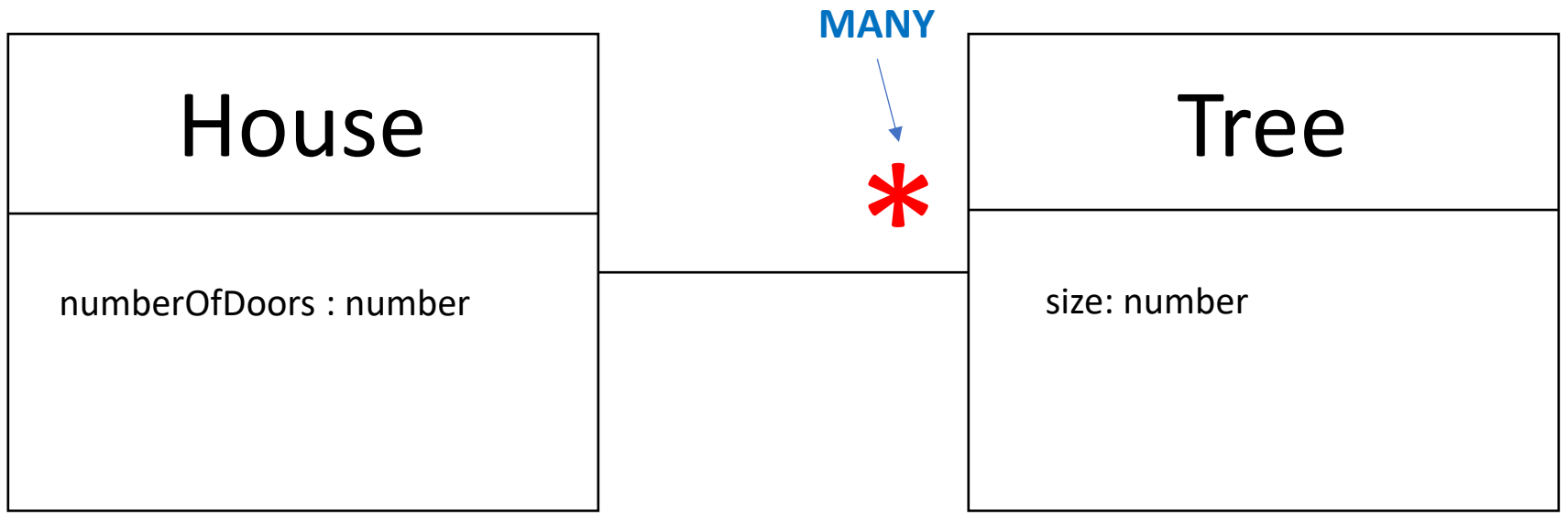
A house can have **MANY** **doors** and **trees**

Each **tree** has a specific **SIZE**





We represent this situation with the bellow **UML Class diagram:**





In **DART**, we represent these 2 classes as follows:

```dart
class House {
  int numberOfDoors;
  List<Tree> trees;

  House(this.numberOfDoors, this.trees);
}
```

```dart
class Tree {
  double height;


  Tree(this.height);
}
```







- Where are the **constructors**?
- Where are the **class attributes**?

I EXPLAIN

Define and **use** classes in Dart

DEFINE CLASS

All classes *except Null* descend from **Object**.

class →

```
class House {
  int rooms;
```

Class Attribute
By default public…
Can be modified outside the class!

Constructor →

```
House(this.rooms);
```

Class Method →

```
  bool isBig() {
    return this.rooms > 5;
  }
}
```

*USE CLASS - **INSTANCIATION***

Create a house With 2 rooms

Create a house With 10 rooms

```
main() {
  House frenchHouse = House(2);

  House khmerHouse = House(10);
  khmerHouse.rooms = 5;
```

We can change x attribute here … as it s public

```
  print(khmerHouse.isBig());
}
```

We invoke a method on an object

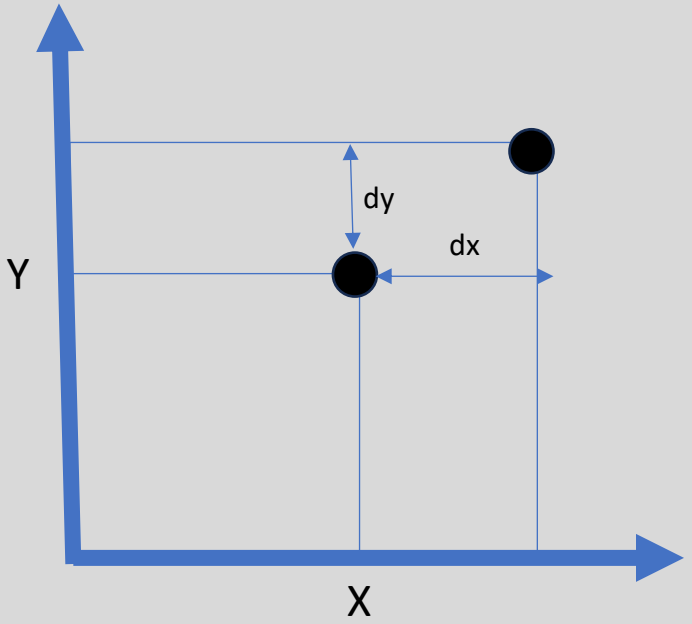


- What will this code output?

I DO

Create a class **Point**

- ✓ A point has a **X and Y** position
- ✓ The class should provide a **method to translate** the point (dx, dy)



OBJECTIVES

- ✓ Create a **class** with attributes
- ✓ Define the **constructor**
- ✓ Add **methods**

SOME HELP !

https://dart.dev/language/classes

CORRECTION

```dart
class Point {
  int x;
  int y;

  Point(this.x, this.y);

  @override
  String toString() {
    return "x= $x - y= $y";
  }
   void translate(int dx, int dy) {
    x+=dx;
    y+-dy;
   }
}

Run | Debug
main() {
  Point p1 = Point(1, 2);
  print(p1);
  p1.translate(10, 10);
  print(p1);
}
```

- ✓ Create a **class** with attributes
- ✓ Define the **constructor**
- ✓ Add **method**

I EXPLAIN

3 way to manage class **constructors**

```
Point(int x, int y) {
                this.x = x;
                this.y = y;
  }
```

Constructor body

Initializer list

```
Point(int x, int y): this.x=x,  this.y=y;
```

```
Point(this.x, this.y);
```

Short form constructor !

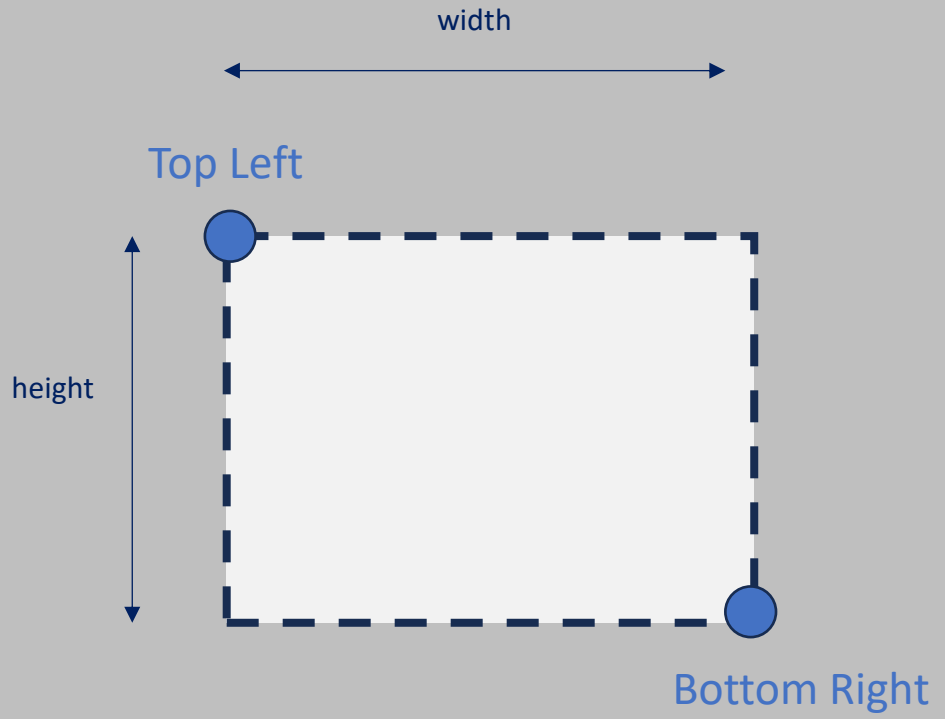
WE DO

Create a class **Rectangle**

- ✓ A Rectangle take 2 points :
  - top-left
  - bottom- right
- ✓ The class provide
  - the rectangle **area**
  - The rectangle **width**
  - The rectangle **height**



Can you handle when the 2 points are not provided in the good order?



OBJECTIVES

- ✓ **Aggregation** of classes
- ✓ **Computed data** with methods
- ✓ Understand getter in Dart

CORRECTION

```dart
class Rectangle {
  Point topLeft;
  Point bottomRight;

  Rectangle(this.topLeft, this.bottomRight);

  // Assuming points are well defined
  int get width => bottomRight._x - topLeft._x;
  int get height => topLeft._y - bottomRight._y;

  int get area => width * height;
}
```

- ✓ **Aggregation** of classes
- ✓ **Computed data** with methods

Getter. As a way to provided **computed data**

CLASSIC GETTER - **FUNCTION SYNTAX**

```
int getWidth() => bottomRight._x - topLeft._x;
```

```
myRectangle.getWith()
```

GETTER WITH **GET KEYWORD**

```
int get width => bottomRight._x - topLeft._x;
```

```
myRectangle.width
```

Width is viewed as a new attribute

YOU DO

Create a class **Person**

A person has:

- A first name
- A last name
- A full name, composed with the first + last name



OBJECTIVES

- ✓ **Computed data** with methods
- ✓ Understand getter in Dart

CORRECTION

```
class Person {
  String firstName;
  String lastName;

  Person(this.firstName, this.lastName);

  String get fullName => this.firstName + " " + this.lastName;
}
```

✓ **Computed data** with methods

YOU DO

Make your Point **immutable !!**

We should NEVER be able to change the class attributes !



Which **key word** can we use to ensure the attribute are **set only once**?



In the method **translate**(dx, dy), we cannot change the attribute value…
-> So what should we do?



OBJECTIVES

✓ Make a class **immutable**

HOW ?

1- Make attributes **final**

2- If you need to change the object attributes=> you return a NEW one

SOME HELP !

✓ Understand immutability

EXPLANATIONS

```dart
class Point {
  final int x;
  final int y;

  Point(this.x, this.y);

  @override
  String toString() {
    return "x= $x - y= $y";
  }

  Point translate(int dx, int dy) {
    return Point(x + dx, y + dy);
  }

}
```

- ✓ Make a class **immutable**

I EXPLAIN

The **4 changes** to make a class **immutable**

MUTABLE

```
class Point {
  int x;
  int y;

  Point(this.x, this.y);

  void translate(int dx, int dy) {
    x+=dx;
    y+-dy;
   }
}
```

IMMUTABLE

```
class Point {
  final int _x;
  final int _y;

  Point(this._x, this._y);


   Point translate(int dx, int dy) {
    return Point(_x+dx, _y+dy);
   }

   int get x => _x;
   int get y => _y;
}
```

1- Make attributes **final**

2- Make attributes **private**

3- Never change object, return a **new one**

3- Provide **getters**



Do we <u>really</u> need private attribute if they are final ?

Named, nullable and required arguments

*What we have learn for **functions** also work for **constructors** !*

Nullable argument

Named argument

Required argument

```
class Shape {
  final int height;
  Color? backgroundColor;

  Shape(
      {
      required this.height,
      this.backgroundColor
      });
}
```

`Shape()`	✗
`Shape(height:10)`	✓
`Shape(height:10, color:Color.blue)`	✓
`Shape(color:Color.blue)`	✗

Enumerations *(enums)*

A type of class accepting only a fixed set of class instance

```
enum Color { blue, red, green, yellow }
```

```
Color myColor = Color.blue;
```

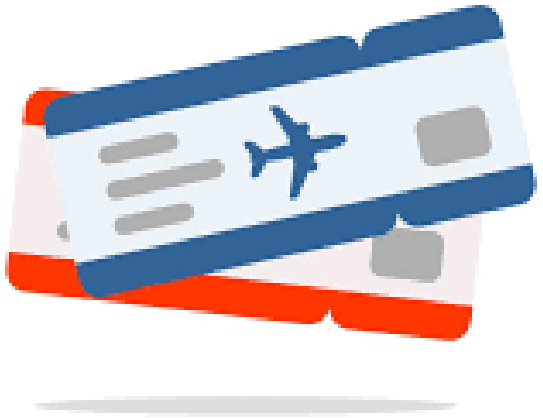
WE DO

Create a class **TravelTicket**

Each ticket contains information about a passenger, destination, travel class, and optional discount code.

1. Complete first bellow table to specify the class attribute:

Attribute	Type	Nullable
Passenger		
Destination		
Travel class		
Discount code		



If a data can contain more than 1 single type of information, a class is needed !

A data that has a fixed set of value, should be an Enum !

2. Code the classes, and the constructors

Create a class **TravelTicket** CORRECTION

Each ticket contains information about a passenger, destination, travel class, and optional discount code.

1. Complete first bellow table to specify the class attribute:

Attribute	Type	Nullable
Passenger	Class	no
Destination	Class	no
Travel class	Enum	no
Discount code	String	yes



If a data can contain more than 1 single type of information, a class is needed !

A data that has a fixed set of value, should be an Enum !

2. Code the classes, and the constructors

EXPLANATIONS

```dart
class Passenger {
  final String name;
  final String? address;

  Passenger({required this.name, this.address});
}

class City {
  final String name;

  City(this.name);
}

enum TravelClass { poor, middle, rich }

class TravelTicket {
  final Passenger passenger;
  final City destination;
  final TravelClass classLevel;
  final String? discountCode;

  TravelTicket({
    required this.destination,
    required this.classLevel,
    required this.passenger,
    this.discountCode,
    });
}
```

We use an enum to represent the travel class type

A class can be composed of other classes !!!

Discount is optional

I EXPLAIN

Named Constructor

A named constructor is a type of constructor that lets you create an object in different ways

SYNTAX

```
ClassName.constructorName(parameters) {
  // constructor body
}
```

EXAMPLE

```
class Student {

  String name;
  int age;

  // 1 - Default constructor
  Student(this.name, this.age);

 // 2 - Named constructor
  Student.guest() {
    name = "Guest Student";
    age = 0;
  }

  // 3 - Another named constructor
  Student.fromMap(Map<String, dynamic>
data) {
    name = data['name'];
    age = data['age'];
  }
}
```

I EXPLAIN

Operator overloading

*Operator overloading allows you to define the **behavior of some operators** (ex +, -)*

```
class Point {
  int x;
  int y;

  Point(this.x, this.y);

  // Overload the + operator
  Point operator +(Point other) {
    return Point(x + other.x, y + other.y);
  }

  @override
  String toString() => "($x, $y)";
}

void main() {
  var p1 = Point(2, 3);
  var p2 = Point(4, 5);
  var result = p1 + p2;
  print(result);
}
```



if point does not extend anything, why toString() have @override ?



What will this code output?

WE DO

Create a class Distance

- ✓ Must be **immutable** !

- ✓ A duration can be entered **in various units**: cms, meters, kms

- ✓ It shall be possible **to add 2 distances** (*and a new distance in return)*
- ✓ We should also **get the distance in various units**: cms, meters, kms



Example of usage:

```
Distance d1 = Distance.kms(3.4);
Distance d2 = Distance.meters(1000);
print((d1 + d2).kms);
```

OBJECTIVES

- ✓ Use **named constructors**
- ✓ Use **getters**
- ✓ Use operator **overloading**

SOME HELP !

- ✓ Understand named constructors
- ✓ Understand getter in Dart
- ✓ Understand operator overloading in Dart

EXPLANATIONS



MASTERS

OF OPERATOR OVERLOAD

LEVEL 4

```dart
class Distance {
  final double _meters;

  Distance.meters(this._meters);
  Distance.kms(double kilometers) : this.meters(kilometers * 1000);
  Distance.cms(double centimeters) : this.meters(centimeters / 100);

  double get kms => _meters / 1000;
  double get cms => _meters * 100;
  double get meters => _meters;

  Distance operator +(Distance other) {
    return Distance.meters(this._meters + other.meters);
  }
}

main() {
  Distance d1 = Distance.kms(3.4);
  Distance d2 = Distance.meters(1000);
  print((d1 + d2).kms);
}
```

This named constructor
Initialize the attributes

The other named constructor
Call the first one



What do you know now





JUNIOR LEVEL

- ✓ Create a **class** with **attributes** and **constructors**
- ✓ Add **methods**
- ✓ Build **aggregation of objects**
- ✓ Create **computed attributes** (get)



NINJA LEVEL

- ✓ Build **immutable** objects
- ✓ Create objects with **optional attributes**
- ✓ Create objects using **different named constructors**
- ✓ Overload the **operator +**